

Build a small LLM from Scratch

Student assignment: Transformer Deep Dive

MISSION

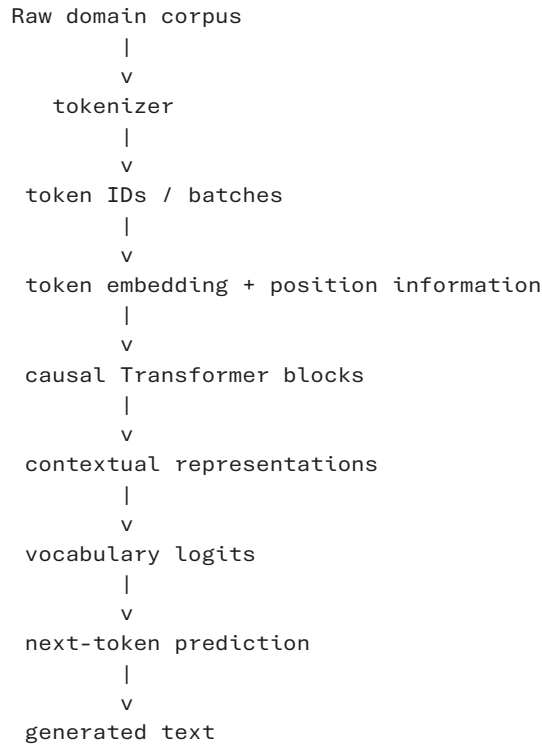
Build, train, inspect, evaluate, and expose a small causal language model using your own Transformer implementation. The final system must generate text from a prompt without loading a pretrained GPT model.

Item	Requirement
Primary objective	Demonstrate end-to-end understanding of Transformer architecture and GPT-style causal language modeling.
Scope	Why Transformers → GPT Causal Language Modeling
Submission	Git repository + technical report + screenshots/evidence + short demo + running Collab demo
Recommended team size	2–3 students
Suggested implementation	Python + PyTorch
Model scale	Small enough to train on your available hardware or Collab; correctness and understanding matter more than model size and performance.

1. Scenario

You are part of a small applied AI team building a text-generation service for a specialized organization. The organization has a private corpus that is too small to justify training a large foundation model, but it is sufficient to teach a compact model the style and vocabulary of a narrow domain.

Your lead engineer asks you to answer a practical question: can a small Transformer trained from scratch learn the next-token structure of a domain corpus well enough to generate coherent text?



What you are building

A miniature GPT-style **causal language model** whose Transformer components are implemented by your thinking and decisions. The system must be reproducible from a clean checkout, and the report must contain evidence that the architecture behaves as intended.

2. Learning objectives

- Explain why Transformers can reduce sequential computation relative to recurrent architectures, including the idea of sequential/parallel depth.
- Derive and implement scaled dot-product self-attention, including masking.
- Implement multi-head attention and track tensor shapes through every transformation.
- Explain how positional information is represented and implement at least one positional encoding strategy.
- Assemble a Transformer block with attention, residual connections, normalization, and feed-forward MLP layers.
- Differentiate bidirectional masked language modeling from autoregressive causal language modeling.
- Construct the next-token prediction dataset and loss used by a GPT-style model.
- Train and evaluate the constructed compact language model without loading a pretrained GPT checkpoint.
- Run controlled ablation experiments and support conclusions with measurements.
- Package the trained model behind a simple API using FAST-API and deploy/host it on online platforms (like [render](#) etc.)

3. Required concept coverage

Lesson	You must demonstrate	Evidence required
01 Why Transformers	Sequential vs parallel depth; speed/memory trade-off	Hand analysis + one measured or reasoned comparison
02 Self-Attention	Q, K, V; scaled dot-product attention; masking	Worked numeric example + implementation + unit test
03 Multi-Head Attention	Multiple heads, reshape/split/merge	Tensor-shape trace + attention output
04 Positional Encoding	Order information added to token representations	Visualization + explanation
05 Full Transformer	Attention + FFN + residuals + normalization	Transformer block code + diagram + shape checks
06 BERT MLM	Why bidirectional masking differs from GPT	Mask matrix + short comparison experiment
07 GPT Causal LM	Causal mask; shifted targets; next-token cross-entropy	Training loop + loss curves + text generation

4. Final deliverables

- One public or instructor-accessible Git repository containing the complete implementation.
- A Google Collab Script
- A trained model checkpoint or a reproducible training command that loads the checkpoint either from GitHub or Shareable Google drive.
- A final README with setup, dataset source, training command, inference command, expected output, and known limitations.
- A technical report as [Medium](#) Article (recommended 8–12 pages excluding appendix).
- At least 6 required screenshots/evidence figures listed in Section 12.
- An ablation results table and discussion.
- A 5-10 minute screen-recorded demonstration showing the model running from a clean environment or recorded terminal session.
- Class Presentations

Definition of done

Other individuals should be able to clone your repository, install dependencies, run the provided tests, reproduce the core architecture checks, and generate text using your model without needing undocumented steps.

5. Milestone plan

Milestone	Focus	Checkpoint
M1	Why Transformers + attention from scratch	Numerical attention example and passing self-attention tests
M2	Multi-head attention + positional encoding	Shape-verified Transformer attention module and position visualization
M3	Transformer block + MLM vs CLM	Complete block and mask comparison
M4	GPT training pipeline	Loss decreases on a small overfit test set
M5	MiniGPT	Model trains on full dataset and generates samples
M6	Evaluation + packaging	Ablation study, inference CLI/API, final report

5.1 Milestone 1 — Attention engine

Start with a tiny sequence that can be calculated by hand. Use a 3-token sequence and low-dimensional vectors. You must show the complete path from Q and K through scaled scores, softmax weights, and the weighted sum of V.

```
scores = (Q @ K.T) / sqrt(d_k)
weights = softmax(scores)
output = weights @ V
```

Then implement a general self-attention function. Include a causal-mask option. Your test suite must verify shapes and must include a case where future positions are forbidden from receiving attention.

5.2 Milestone 2 — Multi-head attention and position

Extend the attention engine to multiple heads. Your implementation must explicitly show attention working within each head, concatenation, and final projection with correct dimensions.

Implement sinusoidal positional encoding. For extra credit only, you may also implement a rotary or learned alternative and compare the behaviors.

5.3 Milestone 3 — Transformer block + objective comparison

Assemble a Transformer decoder-style block appropriate for causal language modeling. In the report, compare it with the bidirectional masking idea used in masked language modeling. You are not required to build a full BERT model; the evidence is intended to prove that you understand why BERT and GPT use different attention constraints.

5.4 Milestone 4 — Training pipeline

- Create contiguous token sequences of a fixed context length.
- Construct shifted inputs and targets: the target at position t is the next token after the input token at position t .
- Use cross-entropy over vocabulary logits (Just a suggestion, You can come up with better options).
- Overfit a deliberately tiny training subset first; use this as a debugging gate before long training.
- Log training loss, validation loss, learning rate, step/epoch, and at least one sample generation during training or evaluation.

5.5 Milestone 5 — MiniGPT training

Train a compact model on the selected corpus. Record the final parameter count, training configuration, hardware used, wall-clock training time, final training loss, final validation loss, and at least five generated samples from fixed prompts.

5.6 Milestone 6 — Evaluation and packaging

Run the ablation study, write the technical interpretation, expose inference through a command-line interface, and provide a small REST API and host/deploy the project on an online platform. The packaging component is part of the engineering grade, not a cosmetic extra.

6. Required hand calculations

Before relying on PyTorch, demonstrate that you can compute the core operation manually. Put the following into your report.

1. Use a 3-token sequence with $d_k = 2$ and $d_v = 2$. Provide explicit Q, K, and V matrices.
2. Calculate QK^T .
3. Divide by $\sqrt{d_k}$.
4. Apply row-wise softmax.
5. Multiply the attention weights by V.
6. Add a causal mask that prevents token 1 from seeing tokens 2 and 3, token 2 from seeing token 3, etc. Show the masked score matrix.
7. Explain in words what changes in the information available to each position.

Do not skip this step

A library call that returns an attention tensor is not evidence that you understand attention. The hand calculation is a required artifact and will be graded separately.

7. Implementation requirements

Component	Minimum implementation requirement
Tokenizer	A deterministic tokenizer with documented vocabulary size and special-token policy. Character-level or subword tokenization is acceptable.
Embedding	Token embedding that maps token IDs to d_{model} vectors.
Position	Sinusoidal positional encoding implemented by you; learned positions may be added as an experiment.
Self-attention	Q/K/V projections, scaled dot-product attention, softmax, optional mask.
Multi-head attention	Configurable number of heads; split/merge operations explicit in code.
Feed-forward network	Two linear transformations with a nonlinearity. GELU is recommended.
Normalization	Layer normalization or an explicitly justified equivalent.
Residual paths	Residual connections around attention and feed-forward sublayers.
Causal masking	Upper-triangular future-token mask or equivalent implementation.
LM head	Projection from d_{model} to vocabulary logits. Weight tying may be used as an optional experiment.
Loss	Token-level cross-entropy with correctly shifted targets.
Generation	Autoregressive sampling with max length and temperature; top-k or top-p is optional.

8. Constraints and academic rules

- Do not load a pretrained GPT, GPT-2, Llama, Mistral, BERT, or other pretrained language-model checkpoint for the core model.
- Do not use transformers library model classes such as `AutoModelForCausalLM`, `GPT2LMHeadModel`, or equivalent high-level pretrained architectures as the submitted model implementation.

- PyTorch tensors, autograd, optimizers, data loaders, and basic numerical utilities are allowed.
- You may use a tokenizer library, but your report must explain the tokenizer and vocabulary it produced.
- External code may be consulted for learning, but copied implementation must be cited and materially understood. You must be able to explain every major model component during the demo.
- All experiments must be reproducible from a clean checkout using documented commands.
- Never commit API keys, credentials, private datasets, or large generated artifacts unnecessarily. Use .gitignore and environment variables where appropriate.

Allowed engineering shortcuts

Using PyTorch for matrix multiplication, softmax, linear layers, LayerNorm, optimizers, automatic differentiation, and GPU execution is explicitly allowed. The educational constraint is on outsourcing the Transformer/GPT architecture to a pretrained model or a high-level black-box architecture.

9. Dataset options

Choose the following dataset to train your model.

Option	Recommended use	Notes
Tiny Shakespeare	Fastest path to a working model	Small text corpus suited to character or token language modeling; the Hugging Face dataset card reports about 1.11 MB and roughly 40,000 lines.

The Tiny Shakespeare dataset is available on Hugging Face and is described as 40,000 lines of Shakespeare from a variety of plays.

Dataset source: [Hugging Face Tiny Shakespeare](#)

10. Starter repository structure

```

transformer-gpt-assignment/
├── README.md
├── pyproject.toml (or requirements.txt)
├── configs/
│   └── tiny_gpt.yaml
├── data/
│   ├── README.md
│   └── .gitkeep
├── src/
│   ├── tokenizer.py
│   ├── dataset.py
│   ├── attention.py
│   ├── positional_encoding.py
│   ├── transformer_block.py
│   ├── gpt.py
│   ├── train.py
│   ├── generate.py
│   └── utils.py
├── tests/
│   ├── test_attention.py
│   ├── test_mask.py
│   ├── test_shapes.py
│   └── test_overfit.py
├── experiments/
│   ├── attention_visualization.py
│   └── ablation.py

```

```

├── evaluate.py
├── outputs/
│   ├── figures/
│   ├── logs/
│   └── samples/
├── checkpoints/
│   └── .gitkeep
├── report/
│   └── report.pdf

```

You may reorganize the repository, but the submitted README must make it obvious where the model, data pipeline, tests, experiments, and report live.

11. Minimum model configuration

Parameter	Starting recommendation	Must be configurable?
vocab_size	Determined by tokenizer	Yes
context_length	128	Yes
d_model	128 or 256	Yes
n_heads	4	Yes
n_layers	4	Yes
ffn_dim	$4 \times d_{\text{model}}$	Yes
dropout	0.0–0.1	Yes
optimizer	AdamW	Yes
learning rate	Start around $3e-4$ and tune	Yes
batch size	As hardware permits	Yes
temperature	0.8 for a baseline generation demo	Yes

Important

These are starting values, not target answers. Your report must justify the configuration you actually train. If your hardware cannot support the suggested settings, reduce the model and document the change.

12. Required screenshots / expected evidence

Include at least the following evidence in the report. Screenshots must show enough context to prove that the result came from your implementation.

Figure	What to capture	What it should prove
1	Hand-calculated attention example	You understand Q, K, V, scaling and softmax.
2	Causal mask matrix or heatmap	Future tokens are blocked.
3	Multi-head tensor-shape printout	The split/merge logic is correct.
4	Positional encoding visualization	Position information changes representations systematically.
5	Model summary / parameter count	Your actual architecture and scale are clear.

Figure	What to capture	What it should prove
6	Training vs validation loss curve	The model is learning and you monitored generalization.
7	Attention visualization from a trained model	You inspected real behavior rather than only loss.
8	Generation samples	The final model can perform autoregressive generation.
9	Ablation results table/plot	You changed one architectural/training factor and measured its effect.
10	CLI or API inference call	The model is packaged as a usable artifact.

12.1 Example expected terminal evidence

```
$ python -m src.generate --prompt "The future of artificial intelligence" --max-new-tokens 80
```

```
Prompt: The future of artificial intelligence
Generated: The future of artificial intelligence ...
```

```
model parameters: 3.2M
context length: 128
temperature: 0.8
```

Your exact numbers will differ. What matters is that the output records enough information to reproduce and interpret the run.

13. Required experiments

Run at least three controlled experiments. In each experiment, change one major factor while keeping the others as stable as practical.

Experiment	Suggested values	Measure
Depth	2 vs 4 vs 6 Transformer blocks	Validation loss, parameter count, training time
Heads	2 vs 4 vs 8 heads	Validation loss and generation quality
Context length	64 vs 128 vs 256	Validation loss, memory usage, training throughput
Optional: position method	Sinusoidal vs learned	Validation loss and sample quality
Optional: generation strategy	Greedy vs temperature sampling	Qualitative diversity and repetition

Your report must distinguish between observation and interpretation. Example: “Validation loss decreased from X to Y” is an observation. “Increasing depth improved long-range modeling because...” is an interpretation that must be supported by evidence.

14. Evaluation and grading rubric

Category	Weight	Full-credit standard
Conceptual understanding	15%	Correctly explains attention, masking, position, Transformer blocks, and the BERT/GPT objective distinction.
Attention implementation	15%	Self-attention and multi-head attention are implemented correctly and tested.
Transformer architecture	15%	Embedding, position, attention, FFN, normalization, residual paths, and LM head are integrated correctly.

Category	Weight	Full-credit standard
Language-modeling pipeline	15%	Dataset construction, target shifting, causal masking, loss, training loop, and checkpointing work correctly.
Experiments and analysis	15%	At least three controlled experiments with meaningful measurements and evidence-based conclusions.
Generation quality and evaluation	10%	Model generates coherent outputs and results are evaluated using fixed prompts plus quantitative loss/perplexity where appropriate.
Software engineering	10%	Clean structure, tests, reproducibility, configuration, documentation, and usable inference command/API.
Report and demo	5%	Clear report, required figures, concise explanation, and successful live/recorded demonstration.

15. Technical report structure

- 1. Problem and scenario:** What are you trying to build
- 2. Transformer foundations:** Explain the architecture with equations, tensor shapes, and the motivation for attention.
- 3. Implementation:** Explain your tokenizer, model configuration, attention implementation, masking, and training loop.
- 4. Dataset and preprocessing:** Describe source, size, split, tokenizer, vocabulary, context length, and any filtering.
- 5. Training setup:** Hardware, hyperparameters, optimizer, batch size, number of steps/epochs, checkpoints.
- 6. Results:** Loss curves, sample generations, parameter count, training time, and attention inspection.
- 7. Ablation study:** Tables/plots and interpretation for the required experiments.
- 8. Failure analysis:** Show at least two failure modes or limitations and explain likely causes.
- 9. Engineering packaging:** Explain CLI/API, reproducibility, and how another user runs inference.
- 10. Conclusion:** Summarize what the experiments demonstrate about Transformer-based causal language modeling.

16. Submission acceptance tests

Before submission, run the following checks from a clean environment or container.

1. Install dependencies
2. Run unit tests
3. Run a tiny overfit test
4. Load the final checkpoint
5. Generate from three fixed prompts
6. Verify that generation never attends to future tokens
7. Verify README commands match the actual repository and confirm no secrets are committed.

Test	Pass condition
Unit tests	All required tests pass without manual edits.
Tiny overfit	Training loss falls substantially on a tiny repeated batch, demonstrating that gradients and target shifting work.
Checkpoint reload	A fresh process can load the saved weights and generate text.
Causal correctness	A token at position t cannot use information from positions $> t$.
Reproducibility	The README includes exact commands and configuration needed to run inference.

17. Demo script (5–7 minutes)

1. Show the repository structure and identify the core model file.
2. Show the attention implementation and causal mask.
3. Run the test suite or key architecture checks.
4. Show the final model configuration and parameter count.
5. Show the training/validation loss curve.
6. Run one generation command from a fixed prompt.
7. Show one attention visualization and explain what you observe.
8. Show deployed solution with live inferencing.
9. Summarize the most important ablation result and one limitation.

18. Collaboration, attribution, and responsible use

- Each team member must understand the submitted model. The demo may include individual oral questions.
- When using external code, tutorials, repositories, equations, or figures, cite the source in the report and identify what was adapted.
- Do not include private, personal, confidential, or licensed material in a dataset unless you have permission to use it.
- Generated text from the trained model is an experimental artifact, not a claim that the model is factually reliable.
- Keep the project reproducible and avoid committing large checkpoints unless explicitly requested by the instructor.

19. Final submission checklist

- ☐ Repository link/access is correct.
- ☐ README setup commands work.
- ☐ Hand-calculated attention is included.
- ☐ Causal mask is shown.
- ☐ Multi-head shape trace is included.
- ☐ Positional encoding evidence is included.
- ☐ Model architecture diagram is included.
- ☐ Training/validation curves are included.
- ☐ At least five generation samples are included.
- ☐ Three required ablations are complete.
- ☐ Final parameter count and training configuration are documented.
- ☐ CLI/API inference works.
- ☐ Technical report is submitted as Medium Article.
- ☐ No secrets or unnecessary large artifacts are committed.
- ☐ All sources are cited.

20. References and starting resources

Assignment curriculum: [Open resource](#) — Lesson sequence used to frame this assignment.

Self-attention from scratch: [Open resource](#) — Reference lesson for the attention implementation milestone.

Multi-head attention: [Open resource](#) — Reference lesson for head splitting and merging.

Positional encoding: [Open resource](#) — Reference lesson for position information.

Full Transformer: [Open resource](#) — Reference lesson for assembling the architecture.

BERT masked language modeling: [Open resource](#) — Reference lesson for bidirectional masked objectives.

GPT causal language modeling: [Open resource](#) — Reference lesson for autoregressive next-token prediction.

Tiny Shakespeare dataset: [Open resource](#) — Recommended starter corpus; the dataset card describes a roughly 1.11 MB corpus with about 40,000 lines of Shakespeare.

Final principle

The goal is not to build the biggest model. The goal is to produce a small, understandable, tested, reproducible Transformer language model and to demonstrate, with evidence, why each architectural component exists.